

BinPoseRT — Real-Time Multi-View 6D Pose Estimation for Robotic Bin Picking

Executive summary

BinPoseRT should be built as a research-grade, mostly software-only perception system for estimating the 6DoF poses of known but potentially unseen industrial objects in heavily cluttered bins. Its strongest version combines RGB-based object discovery and initial pose estimation with RGB-D geometric refinement, calibrated multi-view fusion, symmetry-aware evaluation, confidence estimation, and optional active viewpoint selection. The system should terminate not in a pretty visualization, but in a robot-usable transformation $T_{\text{robot} \leftarrow \text{object}}$, together with a calibrated confidence score and measured latency.

This is an unusually well-aligned project for an industrial 3D-vision position because the **BOP Challenge 2025 Industrial track explicitly centers on XYZ-IBD, ITODD-MV, and IPD**, all aimed at difficult industrial perception; XYZ-IBD in particular contains textureless, metallic, mostly symmetric objects under severe occlusion and dense bin clutter. ¹ BOP itself exists to evaluate accurate, robust and efficient 6DoF object-pose methods relevant to applications such as robotics. ²

The recommended research question is:

How much can depth-aware geometric refinement, calibrated multi-view fusion, symmetry-aware reasoning, and confidence-driven active perception improve the robustness of zero-shot/model-based 6D pose estimation for textureless industrial objects, while retaining a deployable latency budget?

The recommended development path is deliberately incremental:

single-view baseline → depth refinement → symmetry handling → multi-view fusion → uncertainty → active perception

The first credible baseline should use **CNOS or supplied masks + FoundPose/MegaPose**, because these methods map cleanly onto the “CAD model known, object-specific training undesirable” scenario. FoundPose uses DINOv2 features, rendered templates, template retrieval, feature correspondences and PnP-RANSAC; it requires no object-specific training. ³ MegaPose takes an RGB image, camera intrinsics, object mesh and bounding box, and estimates the object's 6D camera-frame pose; its authors trained the generic model on two million synthetic images containing more than 20,000 objects. ⁴

The strongest engineering contribution should **not** be “I ran MegaPose.” It should be the surrounding system:

novel-object segmentation + pose hypotheses + RGB-D registration + multi-view SE(3) fusion + failure detection

This creates multiple defensible research contributions while keeping the project feasible.

A 16-week implementation at approximately **14–20 focused hours/week** is realistic for a polished portfolio/research version, or roughly **225–320 hours**. The recommended MVP is attainable sooner: BOP loading, baseline pose inference, depth refinement, symmetry-aware scoring and two-view fusion should precede active perception and TensorRT optimization.

No physical hardware is required. The core project can be completed with BOP datasets, synthetic RGB-D generation, calibrated virtual cameras and a simulated robot. A real RGB-D camera is a valuable later validation layer; a physical robot arm is optional rather than essential.

The final repository should demonstrate six things visibly and quantitatively:

Capability	What the reviewer should see
6D pose estimation	CAD meshes correctly aligned to cluttered RGB-D scenes
3D geometry	Depth unprojection, point clouds, normals, registration, $SE(3)$ transforms
Research ability	Baselines, controlled ablations, failure analysis
Industrial robustness	T-LESS + ITODD + XYZ-IBD/IPD evaluation
Systems engineering	Profiling, asynchronous pipeline, C++/CUDA/TensorRT where useful
Robotics relevance	Object pose transformed into robot/world coordinates and used in a simulated pick

The ambitious **stretch target** should be an end-to-end p95 update time below roughly **200 ms for one selected object after detection**, but this is a **BinPoseRT project target, not a literature-defined definition of “real time.”** The project should publish actual median/p95 timings rather than claim “real-time” based only on GPU inference time. Historical BOP work illustrates why: one fast GDRNPP variant reported an average image time of 0.23 s, while newer methods and pipelines make different accuracy/runtime trade-offs on different hardware, so runtime figures are not directly portable between systems.

5

Research scope, goals, and success criteria

The project should be framed as a **model-based industrial pose-estimation system**. For every target object, a CAD/mesh model is available; the system observes one or more calibrated RGB-D views of a cluttered bin and predicts each visible object's rigid transformation.

For camera C and object O ,

$${}^C T_O = \begin{bmatrix} {}^C R_O & {}^C t_O \\ 0 & 1 \end{bmatrix} \in SE(3),$$

where $R \in SO(3)$ represents orientation and $t \in \mathbb{R}^3$ represents translation. BOP uses the same fundamental formulation: a 6D pose maps the object model from its model coordinate system into the camera coordinate system. ⁶

Primary project objective. Given N calibrated RGB-D cameras,

$$\{I_i, D_i, K_i, {}^W T_{C_i}\}_{i=1}^N,$$

plus target meshes $\mathcal{M}_j$, estimate

$${}^W \hat{T}_{O_j}$$

for visible instances, together with:

$$p_{\text{valid},j}$$

representing a calibrated probability or confidence that the predicted pose satisfies a specified downstream error criterion.

The system should support:

- single-view RGB or RGB-D operation;
- multi-view RGB-D operation;
- multiple repeated instances of the same CAD object;
- textureless and reflective parts;
- discrete and continuous rotational symmetries;
- heavy occlusion;
- graceful pose rejection rather than confidently returning catastrophic errors.

Those conditions are not artificial. XYZ-IBD was constructed precisely around reflective, textureless, mostly symmetric industrial objects under severe occlusion and dense clutter, and its authors report substantial performance degradation for contemporary methods compared with easier academic benchmarks. ⁷ T-LESS similarly consists of 30 industry-relevant textureless objects with substantial shape similarities and symmetries. ⁸ ITODD was created specifically for industrial 3D object detection and pose estimation. ⁹

The main research questions should be fixed before implementation so that the repository looks like an experiment rather than feature accumulation.

RQ-A — Does depth refinement materially improve zero-shot RGB pose estimates?

$$\Delta AR = AR_{\text{RGB pose} + \text{depth refinement}} - AR_{\text{RGB pose}}$$

Test this as a function of visibility, depth quality and object symmetry.

RQ-B — Which registration strategy is most robust in clutter?

Compare point-to-plane ICP, robust ICP, Generalized ICP and optionally a robust global initializer such as TEASER++ when the coarse pose is unreliable. Open3D provides point-to-plane ICP and Generalized ICP

building blocks, while TEASER++ was developed for robust point-cloud registration under severe correspondence outliers. ¹⁰

RQ-C — How much does multi-view information improve pose estimation?

Evaluate:

$$AR_{1v}, AR_{2v}, AR_{3v}, AR_{4v}$$

and stratify by object visibility. Multi-view pose estimation is already an established direction: CosyPose jointly reasons about hypotheses across views and performs object-level multi-view optimization, while recent industrial work such as SDT-6D explicitly fuses multiple depth maps for cluttered bin-picking. ¹¹

RQ-D — Can a lightweight confidence model predict failures reliably?

A good system should know when it is wrong. The confidence module should therefore be evaluated as a binary predictor of whether a pose satisfies a predefined BOP/ADD(-S) criterion, not merely as an arbitrary scalar.

RQ-E — Can active view selection reduce ambiguity using fewer views than fixed camera acquisition?

Recent active-pose work explicitly uses pose-hypothesis ambiguity/entropy to trigger next-best-view selection rather than blindly accumulating views. ¹² Your version can be much simpler than such systems while still testing the same fundamental idea.

RQ-F — What accuracy is lost or retained during deployment optimization?

Compare research inference with ONNX/TensorRT versions and report:

$$(\text{accuracy, median latency, } p95, \text{ VRAM})$$

rather than FPS alone.

A good final project should satisfy these **project-defined acceptance criteria**:

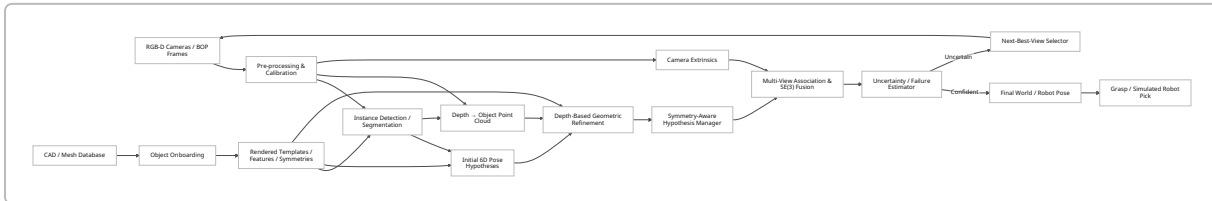
Level	Acceptance criterion
Functional	Full BOP-format RGB-D input → pose → visualization → BOP-compatible result file
Research baseline	Reproduce at least one FoundPose/MegaPose result path on a controlled dataset subset
3D contribution	Demonstrate measurable gain or characterized failure modes from geometric refinement
Multi-view	Two or more camera views fused into one world-frame pose
Symmetry	BOP-compatible symmetry-aware evaluation and fusion

Level	Acceptance criterion
Reliability	Confidence/failure score with calibration analysis
Deployment	Stage-level median and p95 latency; optimized inference path
Standout	Active view selection and simulated pick using final pose

The critical scope discipline is this: **do not train a giant new pose network from scratch**. That would consume most of the project while demonstrating less systems understanding. The more compelling contribution is to take strong general-purpose pose estimators and turn them into a robust industrial perception architecture.

System architecture and algorithm design

The recommended architecture separates perception, geometry, fusion and decision-making. That separation is useful experimentally because every stage can be disabled for ablations.



A suitable internal pose record would be:

```

PoseHypothesis
  object_id
  instance_id
  camera_id
  T_camera_object : 4x4 float64
  score_pose      : float
  score_mask      : float
  visible_fraction : float
  depth_coverage  : float
  icp_fitness     : float
  icp_rmse        : float
  reproj_error    : float
  symmetry_class
  covariance / hypothesis_set
  timestamp
  
```

That becomes much easier to test than passing anonymous matrices between Python scripts.

Detection and instance segmentation. There are two strategically different options.

For a **zero-shot/model-based system**, CNOS is highly appropriate. It uses Segment Anything proposals and DINOv2-based matching against CAD-derived representations and is explicitly designed to segment novel CAD-described objects without retraining. ¹³ FoundPose itself assumes per-instance masks and object IDs and uses CNOS in its experiments. ³

For the **deployment branch**, train a lightweight target-specific detector/segmenter on BOP and synthetic images. That sacrifices zero-shot purity but gives you a useful engineering experiment:

zero-shot segmentation vs target-specific real-time segmentation.

Do not hide this distinction. It creates a useful paper-style trade-off between onboarding cost and runtime.

Initial pose estimation. I would implement three selectable back ends.

FoundPose should be the principal research baseline. It builds object representations from DINOv2 patch descriptors extracted from rendered templates. At query time it retrieves visually similar templates, creates 2D–3D correspondences, applies PnP-RANSAC and then performs featuremetric refinement. It is model-based and does not require object-specific training. ³

This gives BinPoseRT access to explicit correspondences, which are valuable for diagnosis and uncertainty.

MegaPose should be the second baseline and refinement option. MegaPose estimates novel-object pose using a render-and-compare strategy and consumes RGB, camera intrinsics, CAD mesh and bounding box; depth is optional. ¹⁴ Its generic model was trained with two million synthetic images showing more than 20,000 objects, making it a good example of synthetic training for object-level generalization. ⁴

FoundPose's paper specifically reports complementarity between its featuremetric refinement and additional MegaPose refinement, which makes a combined baseline scientifically defensible rather than arbitrary. ³

SAM-6D or FoundationPose should be optional modern comparators, not mandatory dependencies. SAM-6D provides a zero-shot RGB-D pipeline with separate instance-segmentation and pose-estimation networks. ¹⁵ FoundationPose provides a unified novel-object 6D pose estimation/tracking framework supporting model-based CAD input and model-free reference-image input. ¹⁶ Including one of them strengthens the comparative evaluation, but implementing every available method will dilute the core contribution.

The candidate-method decision table should look like this:

Module	Candidate	Strengths	Weaknesses	Compute class	BinPoseRT initial profiling budget*
Segmentation	Provided GT masks	Isolates pose quality cleanly	Not deployable	Very low	\<5 ms

Module	Candidate	Strengths	Weaknesses	Compute class	BinPoseRT initial profiling budget*
Segmentation	CNOS	Novel CAD objects, no target training	SAM/DINO stack is relatively heavy	High	0.1–1 s
Segmentation	Lightweight trained instance segmenter	Fast, TensorRT friendly	Requires target/ synthetic training	Low	10–40 ms
Initial pose	FoundPose	Training-free; interpretable correspondences; good unseen-object fit	RGB ambiguity; template/ onboarding storage	Medium–high	0.1–0.6 s
Initial pose	MegaPose	Strong generic render-and-compare baseline; novel objects	Rendering/ refinement cost; more complex stack	High	0.2–1.5 s
Initial pose	FoundationPose	Unified estimation/ tracking; strong novel-object formulation	More complex dependency/ deployment path	High	profile, no assumed figure
RGB-D baseline	SAM-6D	Zero-shot RGB-D end-to-end comparator	Large pipeline and several dependencies	High	profile, no assumed figure
Refinement	point-to-plane ICP	Simple, fast local geometry refinement	Requires good initialization	Low	5–30 ms
Refinement	robust point-to-plane ICP	Better rejection of clutter/outliers	More tuning	Low	5–40 ms
Refinement	Generalized ICP	Incorporates local surface structure	More compute and parameter sensitivity	Low–medium	10–60 ms
Coarse registration	TEASER++ + ICP	Robust fallback to severe correspondence outliers	Not necessary when initial pose is already close	Medium	20–150 ms

Module	Candidate	Strengths	Weaknesses	Compute class	BinPoseRT initial profiling budget*
Multi-view	weighted SE(3) fusion	Easy to build, fast, explainable	Requires correct instance association	Very low	\<5 ms/object
Multi-view	object-level bundle adjustment	Joint consistency across views	More complex optimization	Medium	10–100+ ms
Active view	discrete candidate scoring	Simple, deterministic, debuggable	Limited to pre-defined viewpoints	Low–medium	10–100 ms excluding capture/motion
Active view	learned/VLM policy	Research novelty	Scope explosion	High	stretch only

*These are **engineering planning envelopes, not published benchmark claims**. Actual values depend heavily on GPU, image resolution, number of objects, number of hypotheses, renderer and implementation. The repository should replace every one of these estimates with measurements from one documented machine. Cross-paper runtime comparisons are inherently imperfect; even BOP challenge reports separately track accuracy and runtime because implementation and hardware matter. ¹⁷

Depth-based refinement. For each segmentation mask S , unproject valid depth pixels using

$$X = \frac{(u - c_x)Z}{f_x}, \quad Y = \frac{(v - c_y)Z}{f_y}, \quad Z = D(u, v).$$

This produces an observed object cloud

$$P_{\text{obs}} = \{\pi^{-1}(u, v, D(u, v)) : (u, v) \in S\}.$$

Transform model points $p_k \in \mathcal{M}$ with the coarse pose:

$$\hat{p}_k = \hat{R}p_k + \hat{t}.$$

Then solve a local registration update.

Point-to-plane ICP minimizes approximately

$$E(R, t) = \sum_i \rho \left(n_i^\top (Rp_i + t - q_i) \right)^2,$$

where q_i and n_i are a matched observed point and surface normal and ρ is an optional robust loss.

Use a coarse-to-fine schedule:

voxel size: 8 mm → 4 mm → 2 mm
max correspondence: 25 mm → 12 mm → 5 mm
iterations: 10 → 15 → 20

Those values are recommended starting points rather than universal constants; scale thresholds relative to object diameter for datasets with very different object sizes.

The key implementation detail is **visibility-aware model cropping**. Do not align the complete CAD cloud blindly against a partially visible depth segment. Render the CAD model at the current pose, determine the predicted visible surface, and register that visible surface to observed depth. Otherwise backside geometry and self-occluded surfaces can create misleading correspondences.

Depth refinement should be rejected if it increases independent image-space or depth-consistency error. That lets you demonstrate a professional principle: *a refinement stage is allowed to say no*.

Symmetry handling. Industrial components make naïve rotation error dangerous. Consider a cylindrical object with symmetry group

$$\mathcal{S} = \{S_1, \dots, S_m\}.$$

Two poses may be physically equivalent when

$$T_1 \sim T_2 \iff \exists S \in \mathcal{S} : T_1 \approx T_2 S.$$

BOP's modern evaluation explicitly uses symmetry-aware measures such as MSSD and MSPD, rather than relying only on raw pose-parameter differences. ⁶

Internally, choose symmetry representatives using

$$S^* = \arg \min_{S \in \mathcal{S}} d_{SE(3)}(T_{\text{reference}}, TS).$$

For continuous axial symmetries, represent only the observable rotational degrees of freedom or discretize the symmetry orbit densely enough for evaluation/fusion.

This is essential during multi-view fusion. Averaging raw quaternions from two equivalent but differently represented symmetric poses can produce an orientation that corresponds to neither valid hypothesis.

Multi-view fusion. Each camera provides

$${}^{C_i}\hat{T}_O.$$

Known extrinsics give

$${}^W\hat{T}_{O,i} = {}^WT_{C_i} {}^{C_i}\hat{T}_O.$$

The simplest useful fusion method is confidence-weighted translation plus symmetry-aligned quaternion or Lie-algebra rotation fusion.

For nearby estimates, map residual transforms into the Lie algebra:

$$\xi_i = \log(T_{\text{ref}}^{-1}T_i) \in \mathfrak{se}(3)$$

and compute

$$\bar{\xi} = \frac{\sum_i w_i \xi_i}{\sum_i w_i}, \quad T_{\text{fused}} = T_{\text{ref}} \exp(\bar{\xi}).$$

Weights can combine:

$$w_i = f(s_{\text{pose}}, s_{\text{mask}}, r_{\text{ICP}}, v_{\text{visible}}, c_{\text{depth}}).$$

After this baseline works, add joint optimization:

$$T_O^* = \arg \min_{T_O} \sum_i \lambda_i E_i(C_i T_W T_O),$$

where E_i includes rendering, depth or feature consistency. CosyPose demonstrates the broader value of jointly reasoning over multi-view object hypotheses and refining them with object-level optimization. 18

Instance association matters when a bin contains five copies of the same component. Use geometric compatibility between candidate world-frame centers, rendered-mask consistency and one-to-one assignment via Hungarian matching. Reject association when uncertainty is too large rather than forcing every detection into a track.

Uncertainty and failure prediction. Do not begin with a complex Bayesian network. Store a feature vector

$$z = [s_{\text{seg}}, s_{\text{pose}}, e_{\text{PnP}}, f_{\text{ICP}}, r_{\text{ICP}}, c_{\text{depth}}, v_{\text{visible}}, e_{\text{silhouette}}, e_{\text{multiview}}, \sigma_{\text{hyp}}].$$

Define success label

$$y = \mathbf{1}[e_{\text{pose}} < \tau]$$

on validation data and fit a small logistic/MLP calibration model:

$$P(y = 1|z) = \sigma(w^\top z + b).$$

This is much easier to defend than inventing a “confidence formula.” Evaluate it with ROC-AUC, precision-recall, Brier score and calibration plots.

Pose-hypothesis dispersion provides an additional uncertainty signal. If $\{T_k\}$ are plausible hypotheses, quantify rotational and translational dispersion after symmetry alignment. Recent active-pose research similarly uses ambiguity across candidate pose hypotheses to decide whether another view is needed. 12

Active-view selection. Start with a discrete set of 20–60 virtual candidate viewpoints around the bin, not learned robot control.

For candidate viewpoint v , score:

$$J(v) = \alpha U(v) + \beta V(v) - \gamma C_{\text{motion}}(v) - \delta C_{\text{collision}}(v)$$

where:

- $U(v)$: expected uncertainty reduction;
- $V(v)$: predicted visible surface fraction or discriminative feature visibility;
- C_{motion} : viewpoint movement cost;
- $C_{\text{collision}}$: invalid/collision penalty.

A particularly clean implementation is to render the current top K pose hypotheses into every candidate view and choose the view where their predicted RGB/depth silhouettes disagree most:

$$U(v) = \frac{1}{K} \sum_k d(R(T_k, v), \bar{R}_v).$$

That transforms next-best-view selection into an information-seeking problem without requiring reinforcement learning. Only after this works should you attempt a learned view policy.

Data, synthetic generation, and calibration

The dataset strategy should not simply concatenate everything. Each dataset should have a specific experimental role.

Dataset	Role in BinPoseRT	Why it matters
T-LESS	Development baseline	30 industry-relevant textureless objects; strong symmetry/shape ambiguity
ITODD / ITODD-MV	Industrial geometry/depth stress test	Industrial parts and machine-vision setting
XYZ-IBD	Primary final industrial test	Reflective, textureless, mostly symmetric, severe clutter; multi-view
IPD	Primary multi-view/HDR industrial test	Real industrial bin-picking with many calibrated camera views
BOP format overall	Unified evaluation/output	Standardized pose tasks, metrics and tooling

T-LESS is a long-standing RGB-D dataset for 6D pose of 30 textureless industry-relevant objects, making it an excellent place to debug symmetries before moving to newer, harder industrial scenes. ⁸

ITODD is explicitly aimed at industrial 3D object recognition and pose estimation. ⁹ For current BOP Industrial evaluation, use the BOP-provided multi-view version where applicable; BOP 2025 grouped **XYZ-IBD, ITODD-MV and IPD** in its industrial track. ¹⁹

XYZ-IBD should become the portfolio centerpiece. Its first release describes **15 textureless, metallic, mostly symmetric objects**, 75 multi-view real scenes, more than 22,000 labeled multi-view RGB-D frames, roughly 273,000 pose annotations and up to 45,000 BlenderProc synthetic views. It was captured with several camera modalities and specifically targets heavy occlusion, reflection and dense industrial bin clutter. ⁷

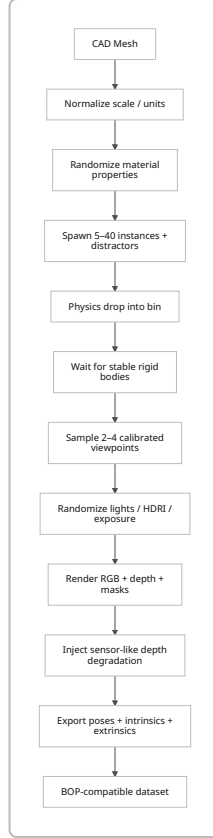
IPD is another particularly appropriate multi-camera benchmark. The IPD work was introduced to co-evaluate cameras, HDR and algorithms for reliable industrial-grade 6DoF estimation, and later multi-view work describes 1,232 physical scenes containing 10 objects observed with 13 multimodal cameras. ²⁰

The correct evaluation hierarchy is therefore:

```
T-LESS
  ↓
debug algorithms and symmetry
  ↓
ITODD
  ↓
industrial geometry/depth
  ↓
XYZ-IBD
  ↓
reflection + clutter + multi-view
  ↓
IPD
  ↓
multi-camera industrial validation
```

Synthetic data should be generated in two stages. BlenderProc is the best first choice because of its BOP-oriented ecosystem; Isaac Sim is more useful later when physics, simulated cameras and robot/viewpoint control become important. NVIDIA's current Replicator/Isaac Sim tooling supports scene-based synthetic-data generation, domain randomization, sensor configuration and structured annotations, and NVIDIA also provides pose-estimation SDG workflows that randomize object and scene properties. ²¹

The synthetic pipeline should create physically plausible piles rather than independently sampling object poses.



Recommended starting randomization ranges are **project choices**, not canonical values:

Variable	Recommended starting range
Objects per scene	5–40, scaled to physical object/bin size
Repeated instances	1–15 per target object
Physics settling	until velocity thresholds remain low for 0.5–1.0 s
Camera count	2–4 primary; generate up to 8 candidate viewpoints for active-view experiments
Elevation	approximately 25° – 75° above bin plane
Azimuth	full 360° , while respecting workspace
Distance	sample so bin occupies roughly 50–90% of image
Focal length	± 10 – 20% around nominal calibration
Light count	1–4 plus optional environment lighting
Exposure	± 1 – 2 EV around nominal
Material roughness	wide range, deliberately include low-roughness metal
Object/base color	random where physically plausible

Variable	Recommended starting range
Background/bin	multiple metallic/plastic shades and roughnesses
Depth noise	z-dependent Gaussian component + edge dropout
Missing depth	random patches and view-angle-dependent dropout
RGB blur	0–2 px equivalent
RGB sensor noise	low-amplitude Poisson/Gaussian mixture
Visibility bins	deliberately balance 10–30%, 30–60%, > 60%
Distractors	geometrically similar objects are more valuable than arbitrary clutter

Do not randomize everything independently to absurd extremes. Synthetic data should approximate the **support of real operating conditions**, not maximize visual chaos.

Generate at least three synthetic regimes:

$$D_{\text{clean}}, \quad D_{\text{moderate-DR}}, \quad D_{\text{strong-DR}}$$

and test:

$$\text{synthetic only} \rightarrow \text{synthetic} + \text{small real subset}$$

to quantify sim-to-real behavior. Large-scale synthetic training is already central to modern pose estimation: MegaPose's released training set has two million synthetic images, and previous BOP challenge results showed strong methods trained entirely on BlenderProc synthetic imagery. ²²

Camera calibration must be part of the project, because multi-view fusion is meaningless if extrinsics are poor.

For camera intrinsics,

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}.$$

Ignoring distortion for notation simplicity,

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} R & t \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}.$$

OpenCV's camera-calibration functions estimate intrinsic and distortion parameters from known calibration targets, while `stereoCalibrate` estimates relative extrinsics for multi-camera systems. ²³

The practical procedure is:

1. Capture many checkerboard/ChArUco poses spanning the complete image, particularly edges and corners.
2. Estimate K and lens distortion independently for each camera.
3. Reject obviously poor frames.
4. Re-optimize intrinsics.
5. Verify reprojection residuals spatially, not just by one global mean.

For a static multi-camera rig, estimate

$${}^{C_1}T_{C_2}, {}^{C_1}T_{C_3}, \dots$$

from common calibration-target observations or from a world-fixed target. Choose one stable world frame W , preferably associated with the bin or robot base.

Then every camera observation can be transformed:

$${}^WT_O = {}^WT_{C_i} {}^{C_i}T_O.$$

Hand-eye calibration is only needed once a physical robot is added. In an eye-in-hand arrangement, robot and camera motions produce equations of the classical form

$$A_i X = X B_i,$$

where X is the fixed transformation between robot gripper and camera. OpenCV includes Tsai, Park, Horaud, Andreff and Daniilidis hand-eye solution methods and documents the associated transformation relationships. ²⁴

For a fixed camera overlooking the robot, estimate

BT_C

directly between robot base B and camera C . The final object pose for manipulation is then:

$${}^BT_O = {}^BT_C {}^CT_O.$$

Calibration quality should be validated independently of calibration reprojection error. Put a known target at several workspace locations and measure:

$$e_t = \|t_{\text{estimated}} - t_{\text{reference}}\|$$

and, for multiple depth cameras, transform their point clouds into W and measure inter-camera surface residuals. A rig can have acceptable 2D reprojection error yet still produce unacceptable 3D disagreement if depth-scale or extrinsic errors remain.

Evaluation and experimental protocol

The primary benchmark should use **BOP's modern symmetry-aware evaluation**, with ADD/ADI retained as supplementary metrics for readability and historical comparison.

BOP currently evaluates pose localization with three complementary errors: **VSD** for visible-surface discrepancy, **MSSD** for maximum symmetry-aware surface distance, and **MSPD** for maximum symmetry-aware projection distance. Average Recall is calculated across thresholds for these errors. ⁶ The official BOP toolkit also implements ADD, ADI and separate translation/rotation errors. ²⁵

For model points $x \in \mathcal{M}$, ADD is:

$$e_{\text{ADD}} = \frac{1}{|\mathcal{M}|} \sum_{x \in \mathcal{M}} \|(Rx + t) - (R^*x + t^*)\|_2.$$

For symmetric objects, ADI uses nearest model-point correspondence:

$$e_{\text{ADI}} = \frac{1}{|\mathcal{M}|} \sum_{x_1 \in \mathcal{M}} \min_{x_2 \in \mathcal{M}} \|(Rx_1 + t) - (R^*x_2 + t^*)\|_2.$$

Because modern BOP metrics have more explicit symmetry handling, **do not use ADD/ADI as the sole headline score**. BOP's task definitions specifically define symmetry-aware MSSD and MSPD and average recall over multiple error thresholds. ⁶

Report at least:

Dimension	Metric
Pose quality	BOP AR: VSD, MSSD, MSPD
Legacy readability	ADD, ADI / ADD-S AUC
Translation	median / 90th percentile $\ t - \hat{t}\ $, mm
Orientation	symmetry-aware angular error
Detection	COCO-style AP or BOP-compatible detection score
Calibration	reprojection RMSE, inter-camera point-cloud alignment
Confidence	ROC-AUC, PR-AUC, Brier score, expected calibration error
Runtime	median, p90, p95 latency by stage and end-to-end
Resources	peak VRAM/RAM
Multi-view efficiency	accuracy versus number of views
Active perception	accuracy versus acquired views and camera-motion cost

The benchmark protocol should be frozen in a YAML file such as:

```

benchmark:
  datasets:
    - tless
    - itodd
    - xyzibd
    - ipd

  pose_metrics:
    bop_ar: true
    add: true
    adi: true
    translation_mm: true
    rotation_deg: true

  visibility_bins:
    - [0.10, 0.30]
    - [0.30, 0.60]
    - [0.60, 1.00]

  latency:
    warmup_frames: 50
    timed_frames: 1000
    cuda_synchronize: true
    batch_size: 1
    report:
      - median
      - p90
      - p95

```

Every experiment should record:

```

git commit
dataset/version
object subset
camera subset
model checkpoint
configuration hash
GPU/CPU
CUDA/TensorRT version
random seed
accuracy metrics
stage timings
peak memory

```

This transforms a GitHub demo into reproducible engineering.

The central ablation matrix should be planned **before** results exist:

ID	Seg.	Initial pose	Depth refine	Symmetry	Multi-view	Confidence	Active view	Scientific question
A0	GT	FoundPose	—	BOP eval only	—	—	—	Upper-bound initial-pose baseline
A1	CNOS	FoundPose	—	BOP eval only	—	—	—	Cost of automatic segmentation
A2	CNOS	FoundPose	point-plane ICP	✓	—	—	—	Does depth refinement help?
A3	CNOS	FoundPose	robust ICP	✓	—	—	—	Does robust loss help clutter?
A4	CNOS	FoundPose	GICP	✓	—	—	—	Registration-method comparison
A5	CNOS	MegaPose	best refine	✓	—	—	—	Initial-estimator dependency
A6	CNOS	FoundPose	best refine	✓	2 views	—	—	Two-view gain
A7	CNOS	FoundPose	best refine	✓	3–4 views	—	—	Multi-view saturation
A8	CNOS	FoundPose	best refine	✓	best	✓	—	Failure detection quality
A9	CNOS	FoundPose	best refine	✓	dynamic	✓	✓	Active vs fixed-view acquisition
A10	fast model	best pose	best refine	✓	best	✓	optional	Accuracy/runtime deployment trade-off

ID	Seg.	Initial pose	Depth refine	Symmetry	Multi-view	Confidence	Active view	Scientific question
A11	synthetic-trained fast model	best pose	best refine	✓	best	✓	optional	Sim-to-real study

A particularly useful detail is to split ICP performance according to **initial pose quality**:

$$e_{\text{initial}} \in [0, 5), [5, 10), [10, 20), [20, \infty) \text{ mm}$$

and likewise for orientation. This will reveal the basin in which refinement helps rather than hides failure.

The most informative final plots would be:

Accuracy-latency Pareto plot.

$$x = \text{p95 latency}, \quad y = \text{BOP AR}$$

with one point for each configuration.

Pose accuracy versus visibility.

10–30% AR	30–60% AR	60–100% AR
--------------	--------------	---------------

This will directly expose whether multi-view fusion solves occlusion rather than merely improving easy examples.

Initial versus refined error CDF.

Plot cumulative translation and symmetry-aware surface error before and after ICP/GICP.

Multi-view scaling curve.

$$\text{views} \in \{1, 2, 3, 4\} \longrightarrow AR$$

with error bars over scenes.

Confidence reliability diagram.

For predictions with reported $p_{\text{valid}} \approx 0.8$, approximately 80% should satisfy your declared success criterion if the confidence is well calibrated.

Risk-coverage curve.

Sort predictions by confidence and show error among accepted predictions as low-confidence predictions are rejected. This is particularly compelling for robotics:

coverage $\downarrow \Rightarrow$ failure rate should sharply decline.

Active perception curve.

Compare:

fixed 1 view, fixed 2 views, random NBV, BinPoseRT NBV

against both accuracy and average number of camera acquisitions.

The final report should include failure galleries categorized as:

```
segmentation failure
pose ambiguity
symmetric equivalent pose
wrong instance association
bad depth / reflection
ICP local minimum
extrinsic calibration error
cross-view conflict
overconfident failure
active-view failure
```

A candidate who can explain **why a system fails** is considerably more convincing than one who reports only an aggregate score.

Implementation, deployment, and hardware strategy

The cleanest project structure is a Python research layer over a C++ geometry/runtime core:

```
binposert/
├─ configs/
│   └─ datasets/
│   └─ models/
│   └─ experiments/
│   └─ calibration/
├─ assets/
│   └─ object_models/
├─ binposert/
│   └─ data/
│   └─ detection/
│   └─ pose/
```

```

|   ├── geometry/
|   ├── multiview/
|   ├── uncertainty/
|   ├── active_view/
|   ├── evaluation/
|   └── visualization/
├── cpp/
|   ├── geometry/
|   ├── registration/
|   ├── runtime/
|   └── bindings/
├── tools/
|   ├── convert_bop.py
|   ├── calibrate_camera.py
|   ├── render_templates.py
|   ├── benchmark.py
|   └── export_onnx.py
├── synthetic/
|   ├── blenderproc/
|   └── isaac_sim/
├── tests/
├── docker/
├── docs/
└── README.md

```

The recommended technology split is:

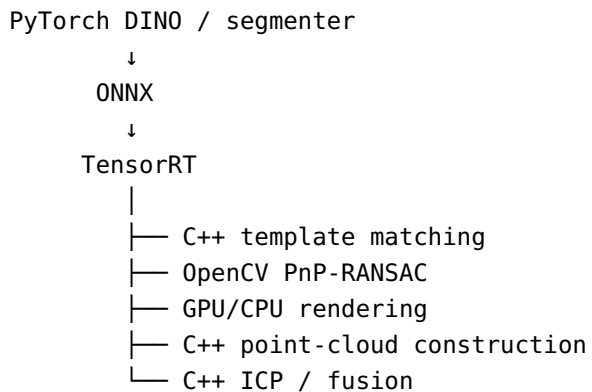
Layer	Technology
Research/orchestration	Python
Neural experimentation	PyTorch
Model interchange	ONNX
Production inference	TensorRT
Geometry/reference	Open3D
Camera/calibration/PnP	OpenCV
High-performance geometry	C++17/20
Custom GPU kernels where justified	CUDA
Configuration	Hydra/YAML or equivalent
Experiment tracking	local JSON/CSV + optionally W&B/MLflow
Reproducibility	Docker

Layer	Technology
Testing	pytest + GoogleTest
Visualization	Open3D + OpenCV
Evaluation	BOP Toolkit

NVIDIA describes TensorRT as a high-performance inference SDK and supports importing model graphs such as ONNX, making the **PyTorch** → **ONNX** → **TensorRT** path appropriate for deployable neural submodules. ²⁶

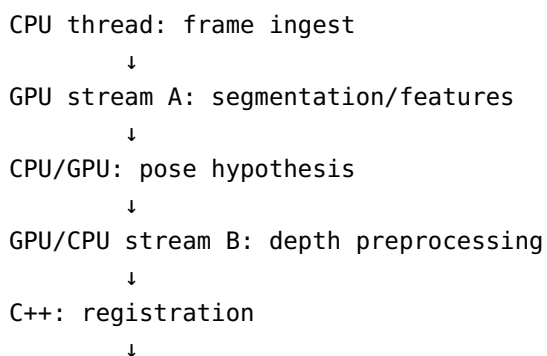
But do **not** promise to export MegaPose or FoundPose end-to-end to one TensorRT engine. Their pipelines contain rendering, template retrieval, correspondence generation, PnP and/or iterative refinement, not merely one feed-forward network. FoundPose's official release, for example, exposes a coarse pose pipeline built around DINOv2 representations, template retrieval and matching rather than a single conventional neural inference graph. ²⁷

Instead optimize by component:



This is more realistic and demonstrates actual systems judgment.

The final runtime should operate as a pipeline:



```
C++: multi-view fusion
      ↓
confidence + publish
```

Profile using explicit CUDA synchronization when reporting GPU timings, because asynchronous kernel launch times are otherwise misleading.

At minimum, report:

$$t_{\text{total}} = t_{\text{pre}} + t_{\text{seg}} + t_{\text{pose}} + t_{\text{depth}} + t_{\text{refine}} + t_{\text{fusion}} + t_{\text{confidence}}.$$

Then report both:

$$\text{median}(t) \quad \text{and} \quad p95(t).$$

A low average latency with occasional one-second spikes is not “real-time” from a robot controller's perspective.

Hardware is optional. The core research project can be completed entirely with prerecorded datasets and simulation:

```
BOP RGB-D
+
virtual calibrated cameras
+
BlenderProc / Isaac Sim
+
simulated robot
```

No physical robot or camera is required.

A sensible hardware ladder is:

Stage	Hardware	Value
Core project	Desktop/laptop + NVIDIA GPU	Everything required for research
Real-scene validation	One RGB-D camera	Tests sim-to-real, calibration, live depth
Multi-view physical test	2–3 RGB-D cameras	Tests real extrinsic calibration and fusion
Robotics stretch	Robot arm	Demonstrates actual eye-to-hand/hand-eye and picking

Stage	Hardware	Value
No robot alternative	Isaac Sim robot	Enough to demonstrate pose-to-manipulation interface

A real sensor adds genuine complications—depth holes, reflections, timing, calibration drift and imperfect segmentation—that datasets cannot fully reproduce. But that validation should be added **after** the BOP pipeline is scientifically solid, not before.

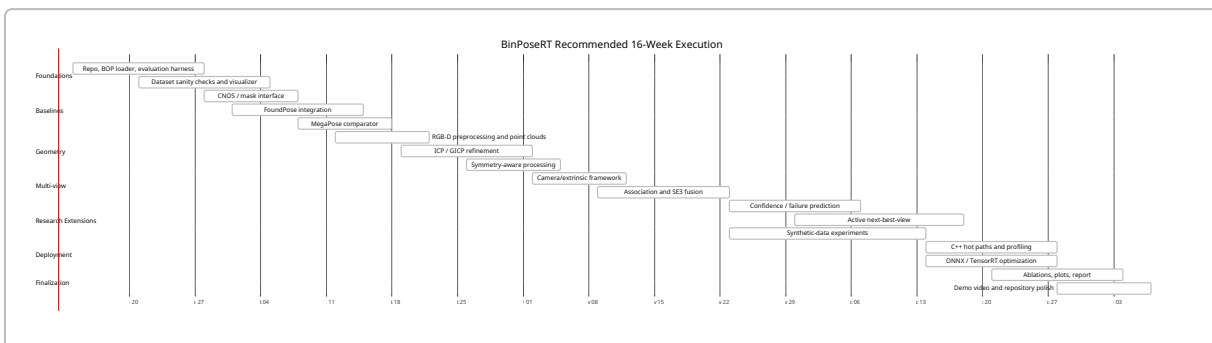
A simulated robot demo should consume precisely the same transform interface as a physical robot would:

$${}^B T_O \rightarrow {}^O T_G \rightarrow {}^B T_G$$

where T_G is a grasp pose. The perception package should therefore remain independent from whichever robot simulator or physical arm is used.

Execution plan, milestones, deliverables, and risks

A sixteen-week schedule gives enough time to build something research-quality without letting active perception consume the entire project.



The dates simply instantiate the plan from mid-September 2026; the relative ordering matters more than the calendar dates.

A realistic effort allocation is:

Phase	Effort	Main exit criterion
BOP/evaluation infrastructure	20–30 h	One command evaluates stored poses
Baseline segmentation + pose	30–45 h	FoundPose or MegaPose results visualized/evaluated
RGB-D geometry/refinement	30–40 h	ICP ablation completed
Symmetry handling	12–20 h	Correct fusion/evaluation on symmetric objects
Multi-view fusion	30–40 h	Two/three-view improvement quantified

Phase	Effort	Main exit criterion
Synthetic generation	20–35 h	Reproducible BOP-format synthetic scenes
Confidence estimation	15–25 h	Reliability/risk-coverage plots
Active-view system	20–35 h	NBV beats random or characterized honestly
Deployment/optimization	25–40 h	stage profiling + optimized path
Final experiments/report/video	25–35 h	publishable repository
Total	227–345 h	complete portfolio system

The project should use explicit gates.

Milestone Alpha — reproducible pose baseline.

```
python tools/run_pose.py \
    dataset=tless \
    pose=foundpose \
    output=outputs/tless_foundpose
```

must produce BOP-compatible predictions and deterministic evaluation.

Milestone Beta — real 3D contribution.

```
python tools/run_pose.py \
    dataset=tless \
    pose=foundpose \
    refinement=robust_icp
```

must produce quantified before/after refinement results.

Milestone Gamma — multi-view.

Two or more calibrated frames must yield one fused object pose in a common world coordinate system.

Milestone Delta — reliability.

Every final pose must include a confidence/failure probability, and the report must show whether that probability is actually calibrated.

Milestone Epsilon — standout version.

On ambiguous or badly occluded cases, the system must either accept the current pose or request a new virtual viewpoint, then quantify whether the extra observation helped.

The final deliverables should be:

Deliverable	Required content
Public code repository	modular Python/C++, configuration, tests
Docker environment	reproducible baseline setup
Dataset tooling	BOP download/convert/config scripts
Synthetic configuration	BlenderProc and/or Isaac Sim scenes/configs
Model assets	checkpoints or documented download scripts
CAD onboarding	template/feature precomputation tool
Calibration module	intrinsics/extrinsics/hand-eye utilities
Benchmark scripts	one-command ablations and timing
Results	CSV/JSON + generated plots
Demo	60–90 second polished video
Technical report	6–10 page paper-style PDF
Model card / limitations	hardware, datasets, known failure modes
Optional hardware demo	live RGB-D scene or robot/simulator

Risk management matters. Several risks are likely enough to plan around.

Risk	Consequence	Mitigation
MegaPose/FoundPose dependency problems	Weeks disappear into environment setup	Dockerize baseline immediately; treat each estimator as a plugin
Pose estimator is too slow	“Real-time” claim fails	Separate accuracy path from fast path; profile before optimizing
Depth hurts instead of helps	Refinement degrades good RGB pose	Visibility-aware CAD rendering, robust losses, refinement acceptance gate
Reflective objects have missing depth	ICP becomes unstable	use RGB pose when depth coverage is low; confidence model incorporates depth coverage
Symmetry corrupts fusion	Multi-view produces invalid orientation	map hypotheses through symmetry group before averaging

Risk	Consequence	Mitigation
Repeated objects are mismatched across cameras	catastrophic fused poses	gated world-space association + Hungarian assignment + rejection
Calibration errors dominate	Multi-view worse than single-view	synthetic perfect-calibration baseline followed by noise sensitivity analysis
Active perception balloons in scope	project never finishes	fixed discrete viewpoints; no RL until baseline complete
TensorRT export fails	deployment stalls	optimize neural submodules only; leave renderer/geometry in C++
Synthetic data fails to transfer	wasted training effort	treat synthetic pipeline as controlled experiment, not prerequisite
Hardware unavailable	demo blocked	simulation remains primary system; hardware is an add-on
Benchmark score mediocre	weak portfolio narrative	emphasize rigorous ablations and honest failure analysis rather than pretending SOTA

One particularly valuable experiment is **calibration sensitivity**. Perturb known camera extrinsics:

$$\delta t \in \{0, 1, 2, 5, 10\} \text{ mm}$$

and

$$\delta \theta \in \{0, 0.1, 0.25, 0.5, 1.0\}^\circ$$

then plot multi-view pose accuracy. This tells a robotics employer that you understand that perception accuracy is bounded by the coordinate system containing it.

Portfolio packaging, README, and demo storyboard

The repository should be written for a technical reviewer who gives it **two minutes initially** and perhaps thirty minutes only if the first two are convincing.

A strong README structure would be:

```
# BinPoseRT
Real-Time Multi-View 6D Pose Estimation for Robotic Bin Picking

## Demo
[short GIF / video]

## What BinPoseRT Does
```

- RGB-D + CAD models → object 6D poses
- depth refinement
- symmetry handling
- multi-view fusion
- confidence / failure detection
- active next-best-view

Key Results

Dataset	AR	Translation error	p95 latency
T-LESS	...		
ITODD	...		
XYZ-IBD	...		
IPD	...		

Accuracy–Latency Plot

System Architecture
[diagram]

Example Results

RGB | mask | initial pose | refined pose | fused multi-view pose

Method

- ### Object onboarding
- ### Segmentation
- ### Initial pose
- ### Geometric refinement
- ### Multi-view fusion
- ### Uncertainty
- ### Active perception

Datasets

T-LESS / ITODD / XYZ-IBD / IPD

Reproducing Results

Running on Custom RGB-D Data

Camera Calibration

C++ / TensorRT Runtime

Ablations

Failure Cases

Project Structure

```
## Installation / Docker

## Citation

## Acknowledgements and Upstream Projects
BOP / FoundPose / MegaPose / CNOS / etc.

## License
```

The “Key Results” block should appear near the top, even if results are not state of the art. It should eventually say something concrete such as:

```
+X.X AR from depth refinement
+Y.Y AR from 3-view fusion
-ZZ% catastrophic failures after confidence rejection
p95 = ... ms on ...
```

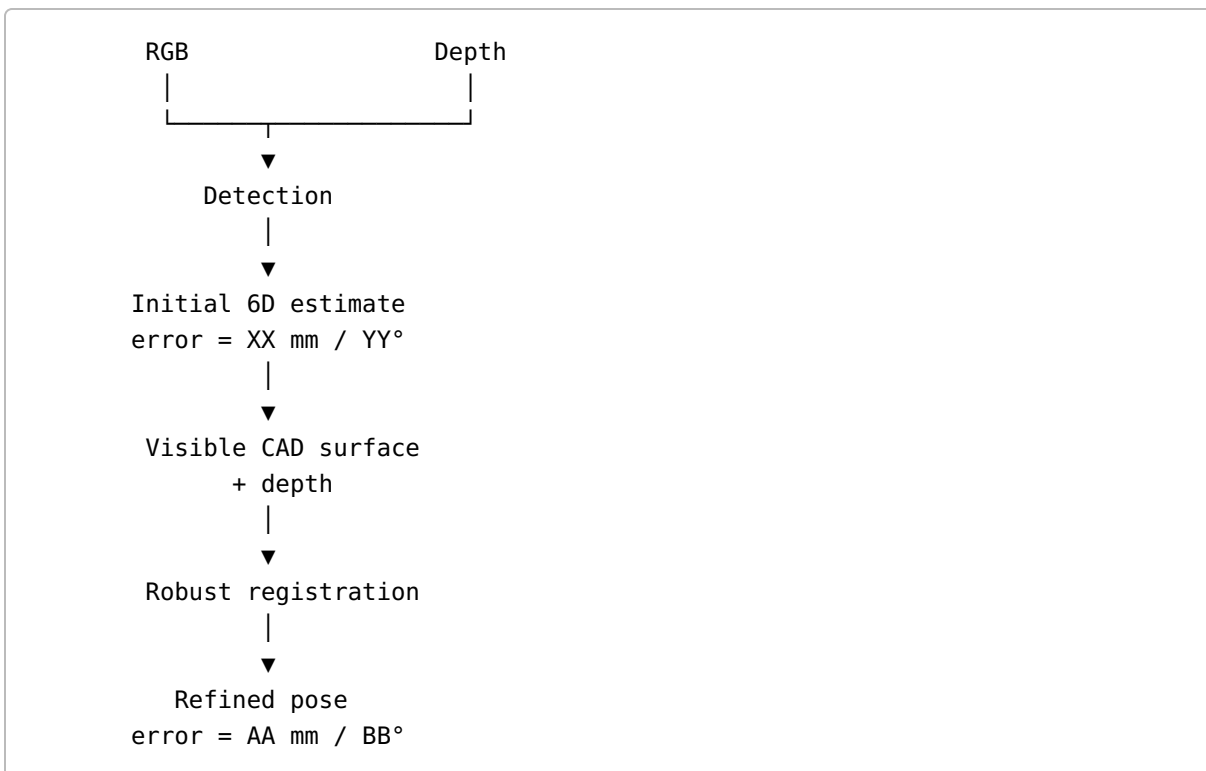
Those numbers must come from your experiments, never from aspirational README writing.

The **60–90 second video** should be storyboarded before filming so it does not become a five-minute screen recording.

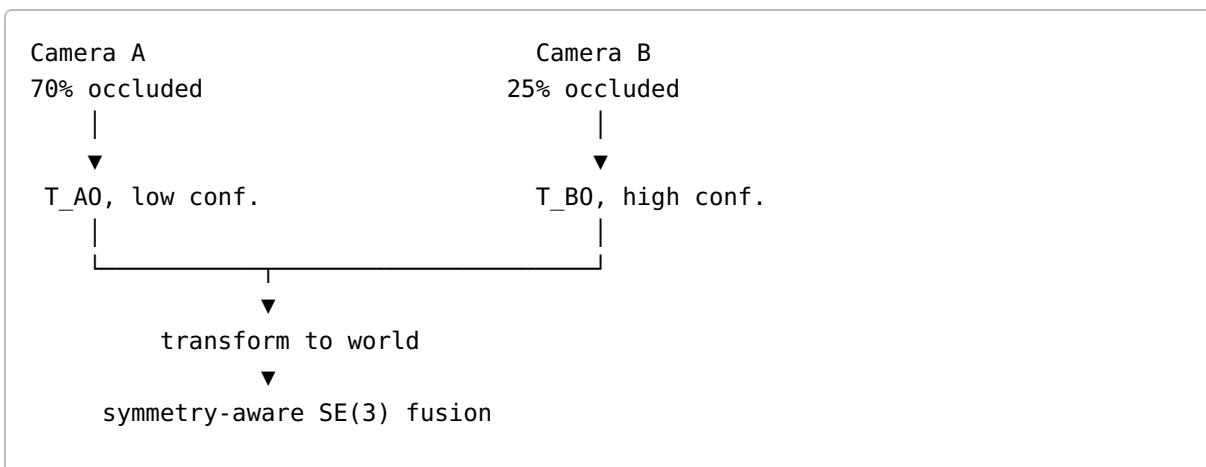
Time	Visual	Message
0–5 s	Dense industrial bin RGB image	“Textureless, symmetric parts. Heavy clutter.”
5–12 s	RGB + depth + 3D cloud	Show that this is genuinely RGB-D/3D
12–20 s	Segmentation masks and CAD identities	Novel-object localization
20–30 s	Initial 6D CAD overlays	FoundPose/MegaPose baseline
30–40 s	Split screen: initial vs depth-refined	Show measurable geometric correction
40–50 s	Two/three camera point clouds aligned in world frame	Multi-view calibration/fusion
50–58 s	One camera heavily occluded, another resolves pose	Explain why multi-view matters
58–66 s	Confidence drops on ambiguous case	“Pose rejected — requesting another view”
66–74 s	Virtual camera moves / next view appears	Active perception

Time	Visual	Message
74-82 s	Correct fused pose + simulated grasp	Close loop to robotics
82-90 s	Benchmark card	AR, mm error, p95 latency, GitHub/ project title

A good visual sequence at the refinement stage is:



For the multi-view shot:



▼
final robot-frame pose

The paper-style report should be written approximately like a workshop paper:

```
Abstract
Introduction
Related Work
System
    Single-view pose estimation
    Geometric refinement
    Multi-view fusion
    Uncertainty
    Active view selection
Experimental Setup
Results
Ablations
Runtime
Failure Analysis
Limitations
Conclusion
```

The most important result is **not necessarily beating the latest leaderboard**. The project is stronger if the conclusions look like:

Robust point-to-plane refinement reduced translation error for sufficiently accurate initial hypotheses but failed when visible depth coverage fell below a certain range; confidence gating recovered most of the lost precision.

or:

Three-view fusion produced its largest gains in the lowest-visibility subset, while gains saturated beyond three views.

or:

Active acquisition matched four fixed views using an average of 2.1 observations.

Those are examples of the **kind** of empirical statement the project should aim to establish, not results to claim before experiments.

The fundamental reason BinPoseRT can stand out is that it connects several research problems that are often demonstrated separately. FoundPose shows that foundation features and template correspondences can estimate unseen-object pose without target-object training; MegaPose shows the effectiveness of generic render-and-compare pose estimation trained on extremely diverse synthetic objects; BOP supplies

standard symmetry-aware evaluation; current industrial datasets such as XYZ-IBD expose exactly the reflective, textureless, symmetric, densely occluded conditions relevant to bin picking; and recent multi-view work confirms that combining views remains an active direction for industrial pose estimation. ²⁸

The resulting project is therefore best described not as “a 6D pose demo”, but as:

BinPoseRT is a reliability-aware RGB-D perception system for model-based 6DoF pose estimation of textureless industrial objects in cluttered bins. It combines zero-shot pose hypotheses, geometry-based refinement, symmetry-aware calibrated multi-view fusion, confidence estimation and active viewpoint selection, and evaluates both accuracy and deployability on BOP industrial benchmarks.

That scope is ambitious, but importantly it is modular: even if the active-view component produces only modest gains, the baseline, geometric refinement, multi-view system, rigorous ablations and deployment work still constitute a substantial project. The BOP ecosystem is particularly suitable for such a project because its benchmark explicitly covers detection, segmentation and 6DoF pose estimation, provides an evaluation toolkit, and as of its latest publicly surfaced challenge information includes a dedicated industrial track centered on datasets directly relevant to robotic bin picking. ²⁹

¹ ² ¹⁹ ²⁹ BOP Challenge 2025

https://bop.felk.cvut.cz/?utm_source=chatgpt.com

³ ²⁷ ²⁸ FoundPose: Unseen Object Pose Estimation with Foundation Features

<https://arxiv.org/html/2311.18809v1>

⁴ ¹⁴ ²² GitHub - megapose6d/megapose6d: Code for "MegaPose: 6D Pose Estimation of Novel Objects via Render & Compare", CoRL 2022. · GitHub

<https://github.com/megapose6d/megapose6d>

⁵ ¹⁷ BOP Challenge 2022 on Detection, Segmentation and Pose Estimation of Specific Rigid Objects

https://arxiv.org/abs/2302.13075?utm_source=chatgpt.com

⁶ BOP: Benchmark for 6D Object Pose Estimation

<https://bop.felk.cvut.cz/tasks/>

⁷ XYZ-IBD: A High-precision Bin-picking Dataset for Object 6D Pose Estimation Capturing Real-world Industrial Complexity

<https://arxiv.org/html/2506.00599v1>

⁸ T-LESS: An RGB-D Dataset for 6D Pose Estimation of ...

https://cmp.felk.cvut.cz/?utm_source=chatgpt.com

⁹ MVTec ITODD | Industrial 3D Object Detection Dataset

https://www.mvtec.com/?utm_source=chatgpt.com

¹⁰ open3d.pipelines.registration — Open3D 0.17.0 documentation

https://www.open3d.org/?utm_source=chatgpt.com

¹¹ ¹⁸ CosyPose: Consistent multi-view multi-object 6D pose estimation

<https://arxiv.org/abs/2008.08465>

- 12 **ActivePose: Active 6D Object Pose Estimation and Tracking for Robotic Manipulation**
<https://arxiv.org/html/2509.11364v1>
- 13 **GitHub - nv-nguyen/cnos: [ICCV 2023 R6D] PyTorch implementation of CNOS: A Strong Baseline for CAD-based Novel Object Segmentation based on Segmenting Anything and DINOv2 · GitHub**
<https://github.com/nv-nguyen/cnos>
- 15 **GitHub - JiehongLin/SAM-6D: [CVPR2024] Code for "SAM-6D: Segment Anything Model Meets Zero-Shot 6D Object Pose Estimation". · GitHub**
<https://github.com/JiehongLin/SAM-6D>
- 16 **FoundationPose: Unified 6D Pose Estimation and Tracking of Novel Objects | Research**
https://research.nvidia.com/publication/2024-06_foundationpose-unified-6d-pose-estimation-and-tracking-novel-objects?utm_source=chatgpt.com
- 20 **Towards Co-Evaluation of Cameras, HDR, and Algorithms ...**
https://ieeexplore.ieee.org/?utm_source=chatgpt.com
- 21 **Scene Based Synthetic Dataset Generation**
https://docs.isaacsim.omniverse.nvidia.com/6.0.1/replicator_tutorials/tutorial_replicator_scene_based_sdg.html?utm_source=chatgpt.com
- 23 24 **Camera Calibration and 3D Reconstruction - OpenCV documentation**
https://docs.opencv.org/4.13.0/d9/d0c/group__calib3d.html?utm_source=chatgpt.com
- 25 **bop_toolkit/bop_toolkit_lib/pose_error.py at master · thodan/bop_toolkit · GitHub**
https://github.com/thodan/bop_toolkit/blob/master/bop_toolkit_lib/pose_error.py?utm_source=chatgpt.com
- 26 **Example Deployment Using ONNX — NVIDIA TensorRT**
https://docs.nvidia.com/?utm_source=chatgpt.com